Digital Logic Design

Prof. Dr. Muhammad Usman
Department of Computer Science
UET Mardan

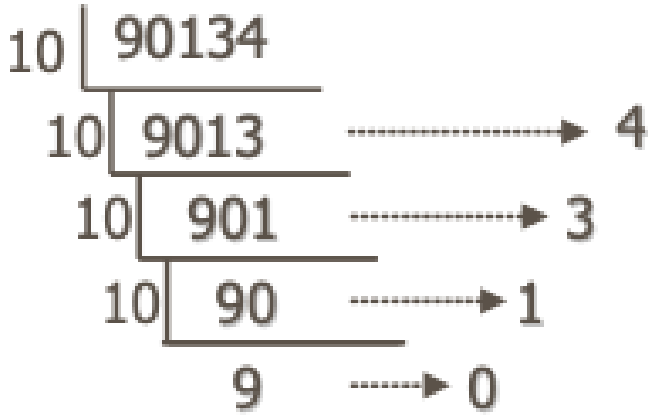
Email: usman@uetmardan.edu.pk

Number Systems

- **Decimal**
  - Base 10
- **Binary**
  - Base 2
- **Octal**
  - Base 8
- **Hexadecimal**
  - Base 16

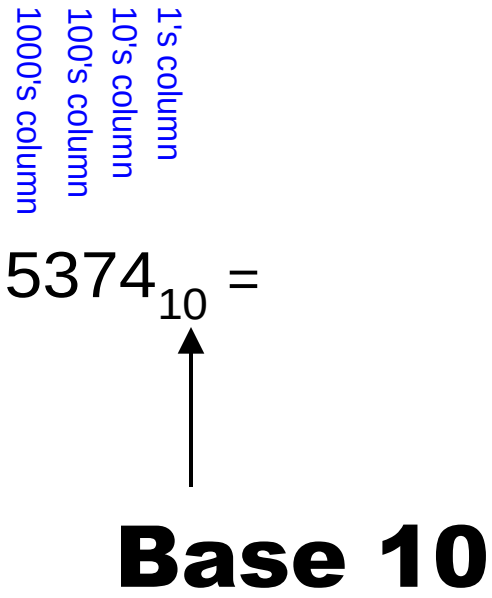
Decimal Number Representation

- Example: 90134 (base-10, used by Homo Sapien)
  - = 90000 + 0 + 100 + 30 + 4
  - = $9*10^4 + 0*10^3 + 1*10^2 + 3*10^1 + 4*10^0$
- How did we get it?



Review: Decimal Numbers

- **Base 10** (our everyday number system)



Review: Decimal Numbers

- **Base 10** (our everyday number system)

1000's column
100's column
10's column
1's column

$$5374_{10} = 5 \times 10^3 + 3 \times 10^2 + 7 \times 10^1 + 4 \times 10^0$$

five thousands | three hundreds | seven tens | four ones

Base 10

Generic Number Representation

- 90134
  - $= 9*10^4 + 0*10^3 + 1*10^2 + 3*10^1 + 4*10^0$
- $A_4\ A_3\ A_2\ A_1\ A_0$ for base-10 (or radix-10)
  - $= A_4*10^4 + A_3*10^3 + A_2*10^2 + A_1*10^1 + A_0*10^0$
  - (A is coefficient; b is base)
- Generalize for a given number N w/ base-b

$N = A_{n-1}\ A_{n-2} \ldots A_1\ A_0$

$N = A_{n-1}*b^{n-1} + A_{n-2}*b^{n-2} + \ldots + A_2*b^2 + A_0*b^0$

**Note that $A < b$

Decimal and Binary Numbers

- **Base 10** (our everyday number system)

1000's column 100's column 10's column 1's column

$$5374_{10} = 5 \times 10^3 + 3 \times 10^2 + 7 \times 10^1 + 4 \times 10^0$$

five thousands | three hundreds | seven tens | four ones

- **Base 2**: Binary numbers

8's column 4's column 2's column 1's column

$$1101_2 =$$

Base 2

Decimal and Binary Numbers

- **Base 10** (our everyday number system)

1000's column 100's column 10's column 1's column

$$5374_{10} = 5 \times 10^3 + 3 \times 10^2 + 7 \times 10^1 + 4 \times 10^0$$

five thousands; three hundreds; seven tens; four ones

- **Base 2**: Binary numbers

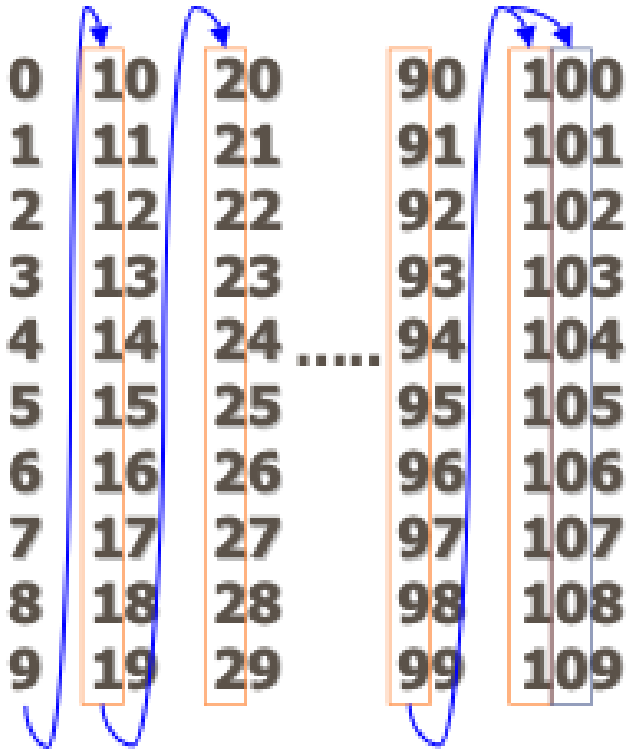
8's column 4's column 2's column 1's column

$$1101_2 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 13_{10}$$

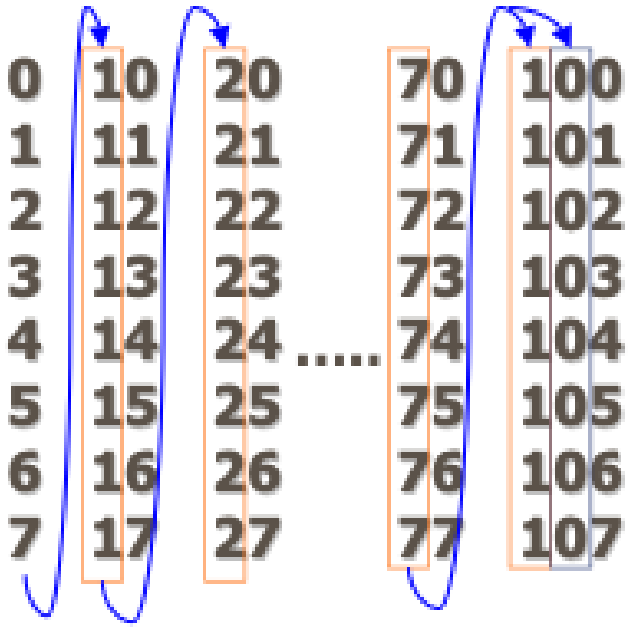
one eight; one four; no two; one one

Base 2

Counting Sequences in Base-b

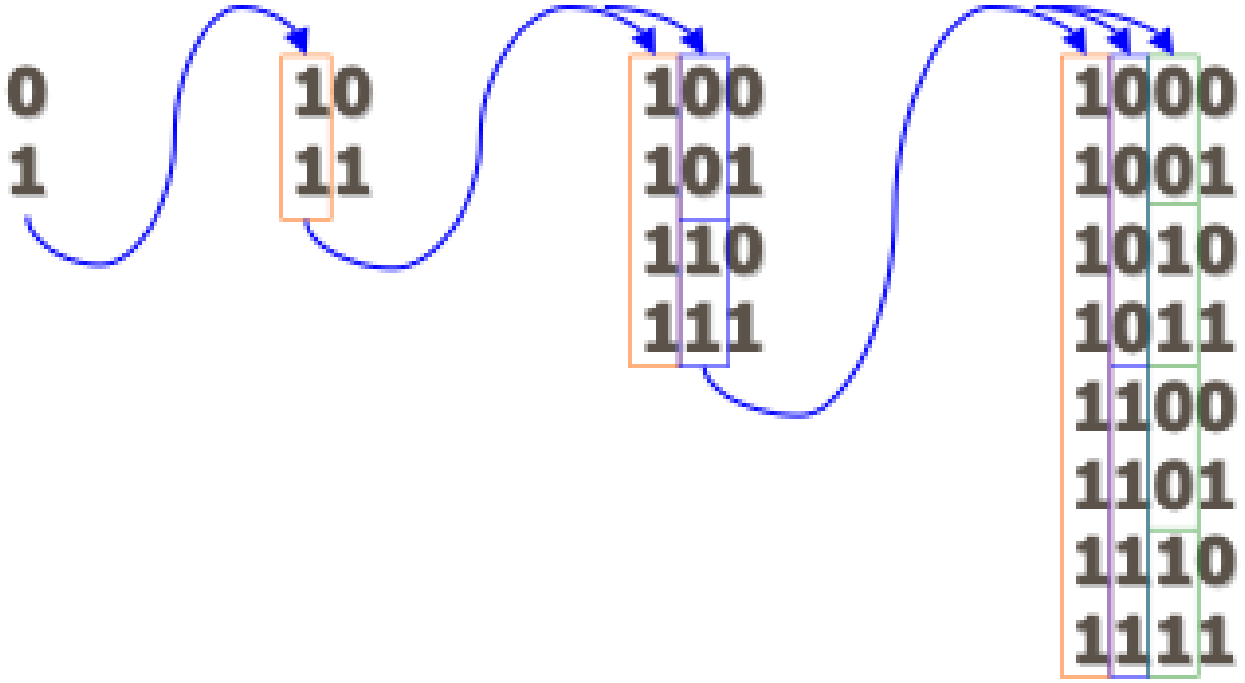


Base-10



How about Base-8

How about base-2



How about base-2

0 = 0	10 = 2	100 = 4	1000 = 8
1 = 1	11 = 3	101 = 5	1001 = 9
		110 = 6	1010 = 10
		111 = 7	1011 = 11
			1100 = 12
			1101 = 13
			1110 = 14
			1111 = 15

Binary = Decimal

- Radix = 2; Digits A_i can only take one of two values (0 or 1)
  - It is customary to refer to *b*inary dig*it*s as *bit*s

	2	3	4	5	···	8	···	10	11	12	···	16
	0001	001	01	01		01		01	01	01		1
	0010	002	02	02		02		02	02	02		2
	0011	010	03	03		03		03	03	03		3
	0100	011	10	04		04		04	04	04		4
	0101	012	11	10		05		05	05	05		5
	0110	020	12	11		06		06	06	06		6
$(N)_b$	0111	021	13	12		07		07	07	07		7
	1000	022	20	13		10		08	08	08		8
	1001	100	21	14		11		09	09	09		9
	1010	101	22	20		12		10	0A	0A		A
	1011	102	23	21		13		11	10	0B		B
	1100	110	30	22		14		12	11	10		C
	1101	111	31	23		15		13	12	11		D
	1110	112	32	24		16		14	13	12		E
	1111	120	33	30		17		15	14	13		F

Powers of Two

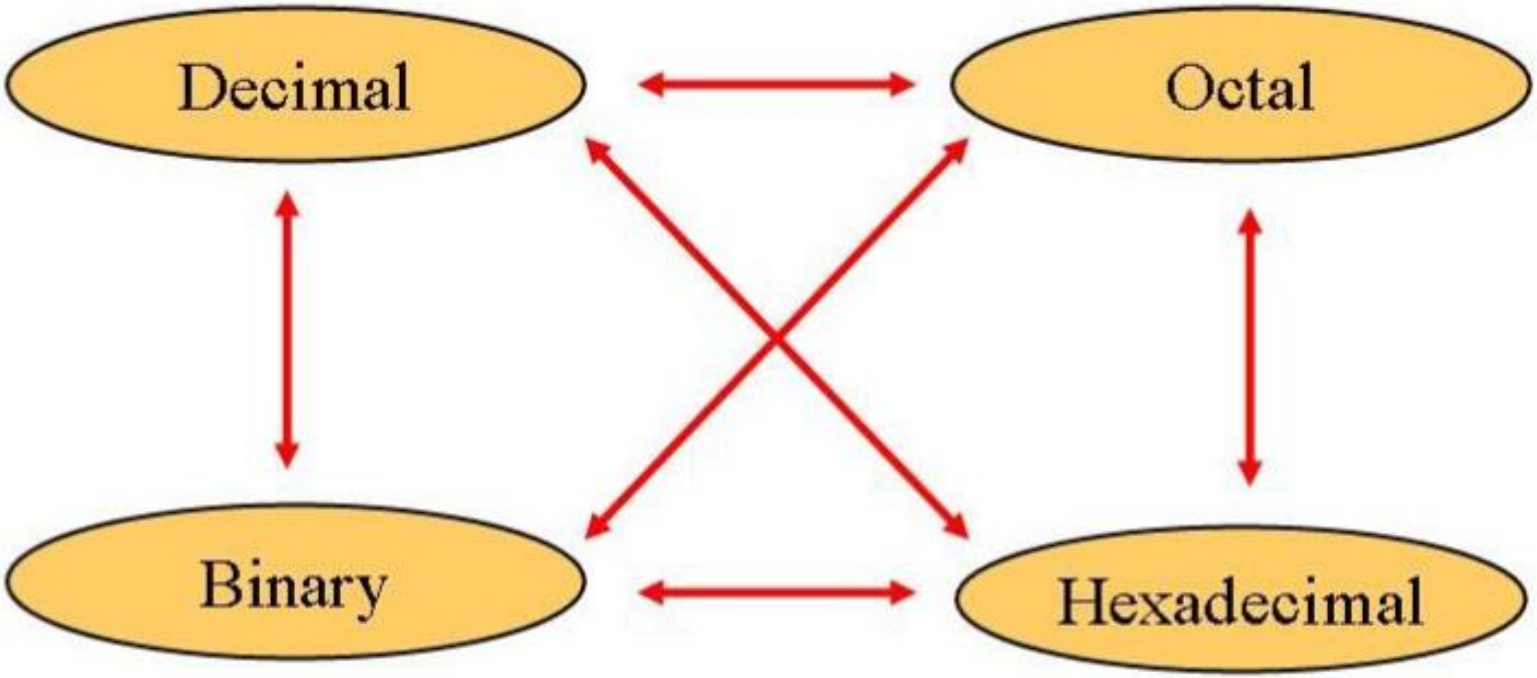
- 2^0 =
- 2^1 =
- 2^2 =
- 2^3 =
- 2^4 =
- 2^5 =
- 2^6 =
- 2^7 =
- 2^8 =
- 2^9 =
- 2^{10} =
- 2^{11} =
- 2^{12} =
- 2^{13} =
- 2^{14} =
- 2^{15} =

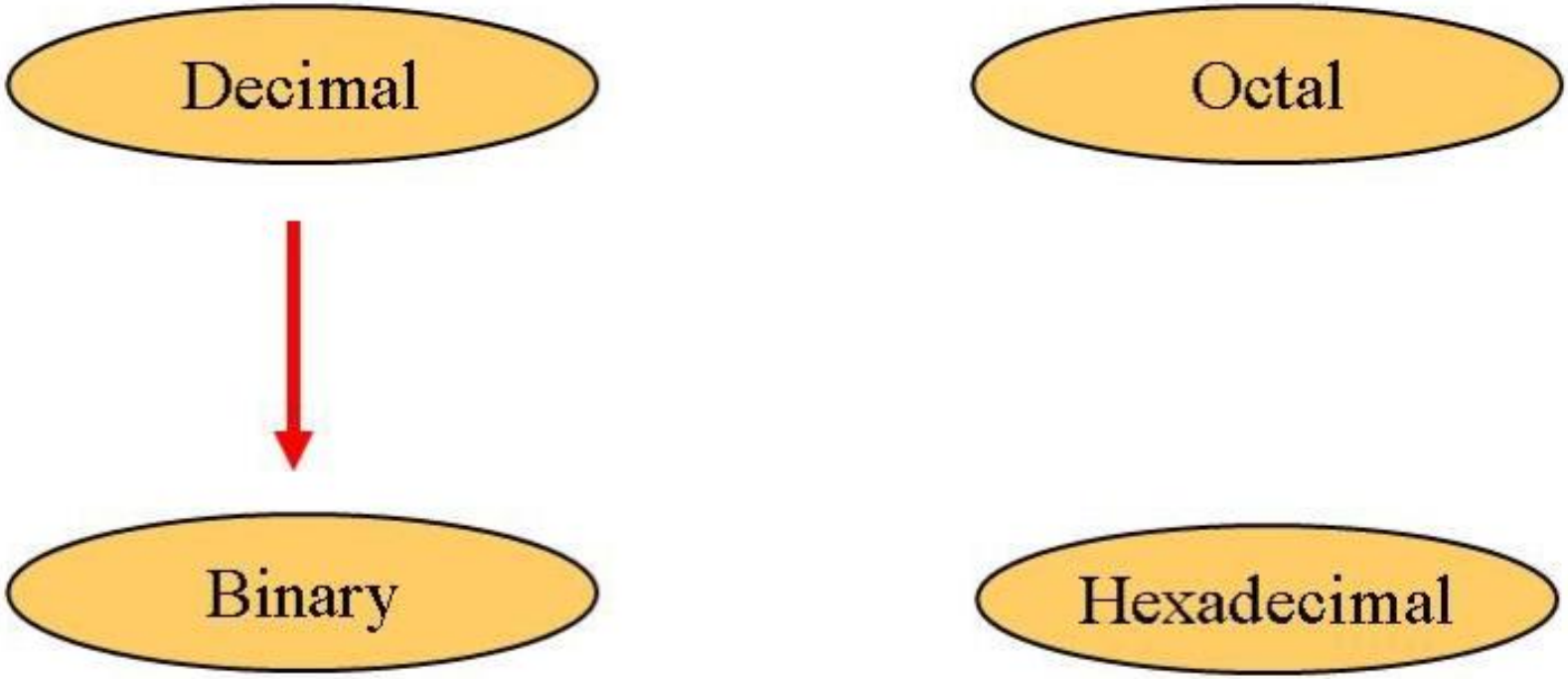
Powers of Two

- $2^0 = 1$
- $2^1 = 2$
- $2^2 = 4$
- $2^3 = 8$
- $2^4 = 16$
- $2^5 = 32$
- $2^6 = 64$
- $2^7 = 128$
- $2^8 = 256$
- $2^9 = 512$
- $2^{10} = 1024$
- $2^{11} = 2048$
- $2^{12} = 4096$
- $2^{13} = 8192$
- $2^{14} = 16384$
- $2^{15} = 32768$
- Handy to memorize up to 2^9

Conversion Among Bases

The possibilities:




Decimal
Octal
Binary
Hexadecimal

Decimal to Binary Conversion

- **Two methods:**
  - **Method 1:** Find the **largest power of 2 that fits**, subtract and repeat.
  - **Method 2:** Repeatedly **divide by 2**, remainder goes in next most significant bit

Decimal to Binary Conversion

Method 1: Find the **largest power of 2 that fits**, subtract and repeat.

53_{10}

Method 2: **Repeatedly divide by 2**, remainder goes in next most significant bit.

Decimal to Binary Conversion

Method 1: Find the **largest power of 2 that fits**, subtract and repeat.

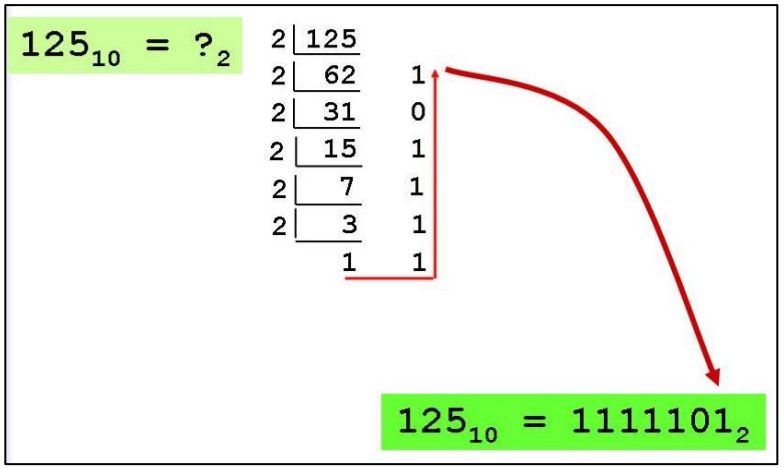
53_{10}	32×1
53-32 = 21	16×1
21-16 = 5	4×1
5-4 = 1	1×1
	= 110101_2

$$625 - 512 = 113 = N_1 \qquad 512 = 2^9$$
$$113 - 64 = 49 = N_2 \qquad 64 = 2^6$$
$$49 - 32 = 17 = N_3 \qquad 32 = 2^5$$
$$17 - 16 = 1 = N_4 \qquad 16 = 2^4$$
$$1 - 1 = 0 = N_5 \qquad 1 = 2^0$$
$$(625)_{10} = 2^9 + 2^6 + 2^5 + 2^4 + 2^0 = (1001110001)_2$$

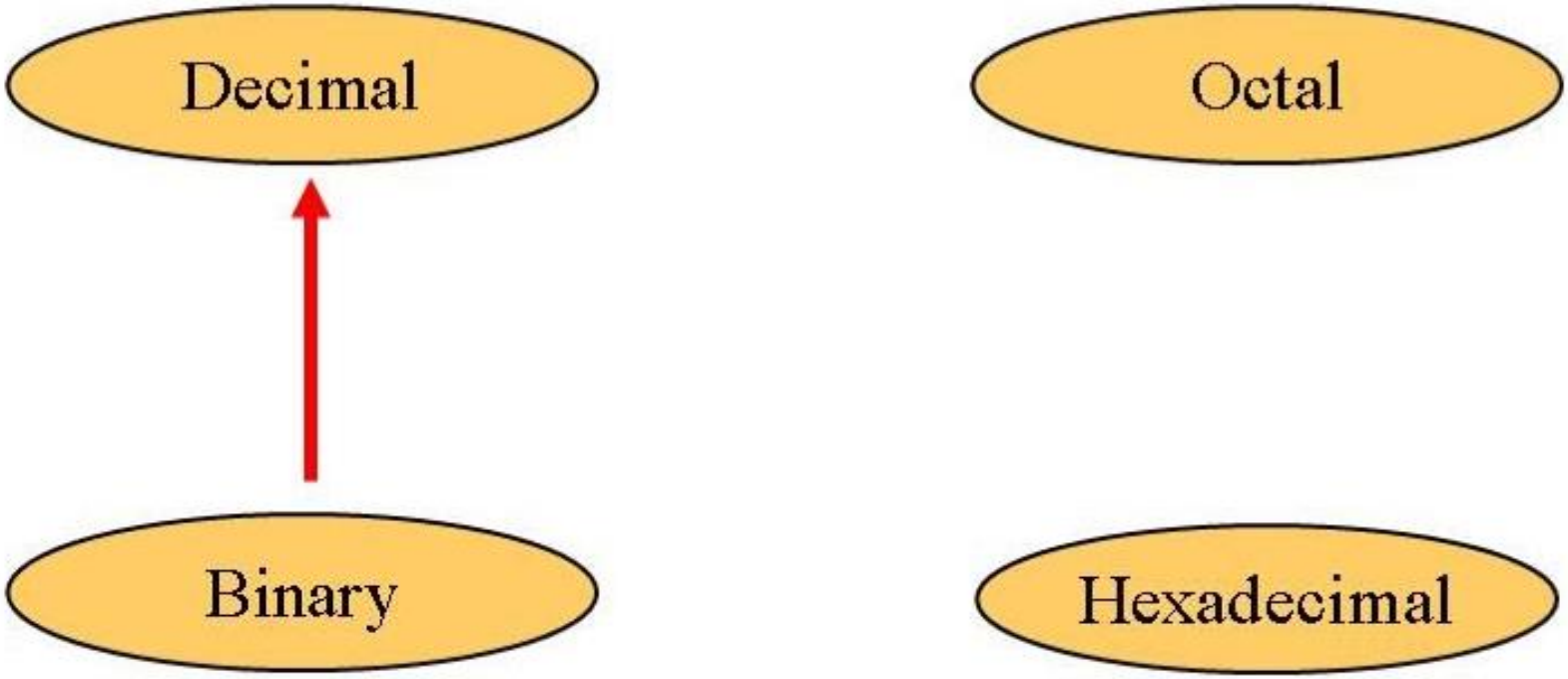
Method 2: Repeatedly **divide by 2**, remainder goes in next most significant bit.

53_{10} =

53/2 = 26 R1
26/2 = 13 R0
13/2 = 6 R1
6/2 = 3 R0
3/2 = 1 R1
1/2 = 0 R1 = 110101_2



Binary to Decimal



Binary to Decimal Conversion

Technique

- Multiply each bit by 2^n, where n is the "weight" of the bit
- The weight is the position of the bit, starting from 0 on the right
- Add the results

Example

- Convert 10011_2 to decimal
- $16\times1 + 8\times0 + 4\times0 + 2\times1 + 1\times1 = 19_{10}$

- **Binary to decimal conversion:**
  - Convert 11101_2 to decimal
  - $16\times1 + 8\times1 + 4\times1 + 2\times0 + 1\times1 = 29_{10}$

Binary Numbers

$$A: \{a_{N-1}, a_{N-2}, \ldots a_1, a_0\}$$

$$A = \sum_{i=0}^{N-1} a_i 2^i$$

Example:

1101_2 $= 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$

$= 8 + 4 + 0 + 1$

$= 13$

- ***N*-digit decimal number**
  - How many values?
  - Range?
  - Example: 3-digit decimal number:

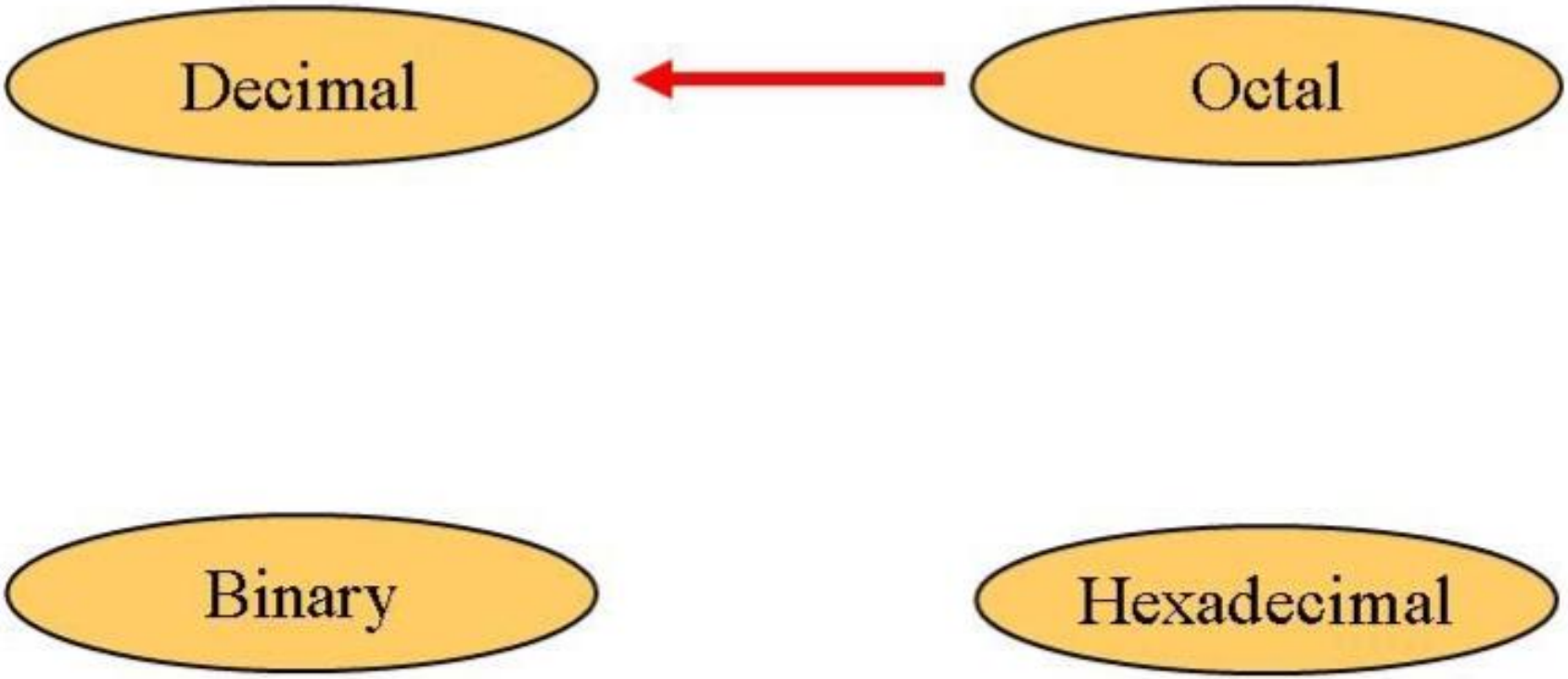
- ***N*-bit binary number**
  - How many values?
  - Range:
  - Example: 3-digit binary number:

Binary Values and Range

- ***N*-digit decimal number**
  - How many values? **10^N**
  - Range? **$[0, 10^N - 1]$**
  - Example: 3-digit decimal number:
    - **10^3 = 1000 possible values**
    - **Range: [0, 999]**

- ***N*-bit binary number**
  - How many values? **2^N**
  - Range: **$[0, 2^N - 1]$**
  - Example: 3-digit binary number:
    - **2^3 = 8 possible values**
    - **Range: [0, 7] = [000_2 to 111_2]**

Octal to Decimal



Octal to Decimal

Technique

- Multiply each bit by 8^n, where n is the "weight" of the bit
- The weight is the position of the bit, starting from 0 on the right
- Add the results

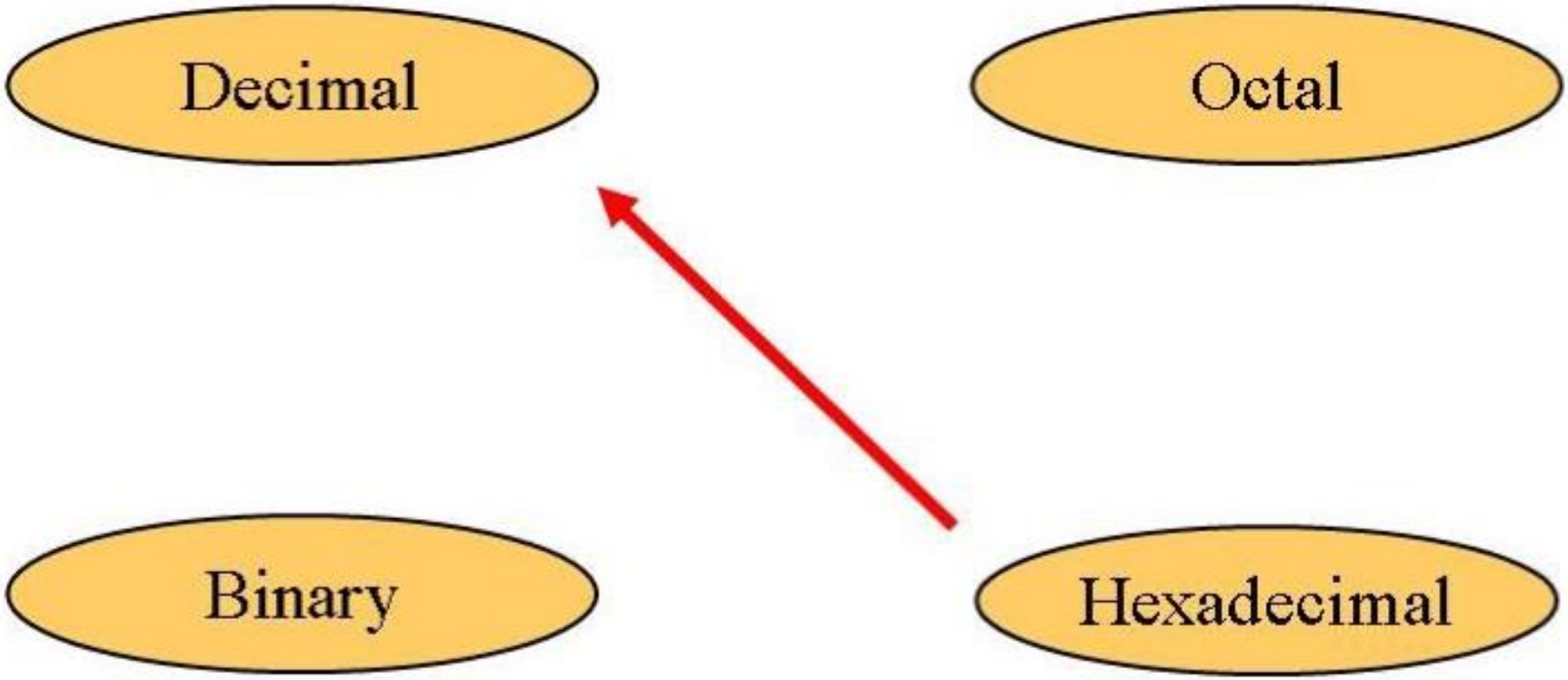
$$724_8 \Rightarrow \begin{array}{rrrrr} 4 & \times & 8^0 & = & 4 \\ 2 & \times & 8^1 & = & 16 \\ 7 & \times & 8^2 & = & 448 \\ \hline & & & & 468_{10} \end{array}$$

Hexadecimal Numbers

- Base 16
- Shorthand for binary

Hex Digit	Decimal Equivalent	Binary Equivalent
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Hexadecimal to Decimal



Hexadecimal to Decimal

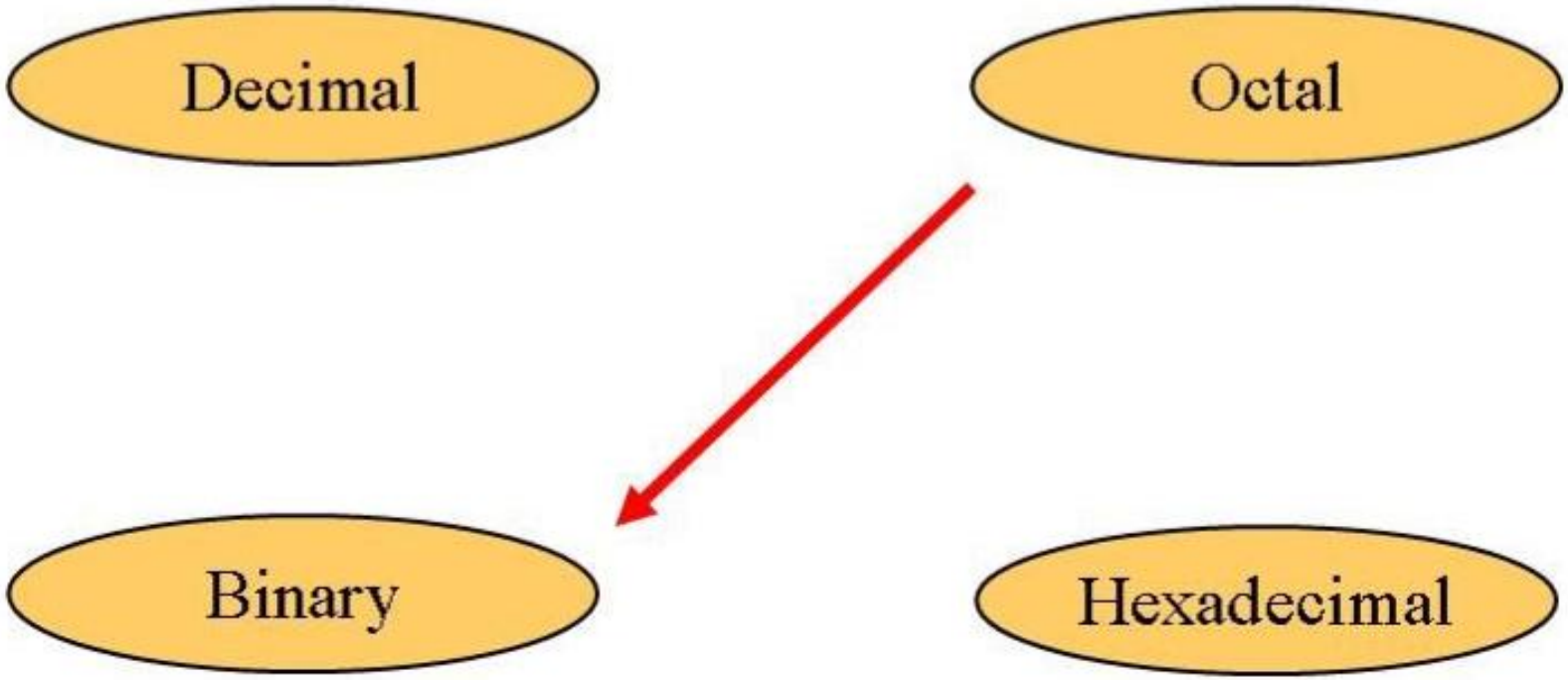
Technique

- Multiply each bit by 16^n, where n is the "weight" of the bit.
- The weight is the position of the bit, starting from 0 on the right.
- Add the results.

$$
\begin{aligned}
ABC_{16} \Rightarrow C \times 16^0 &= 12 \times 1 = 12 \\
B \times 16^1 &= 11 \times 16 = 176 \\
A \times 16^2 &= 10 \times 256 = 2560 \\
&\hline \\
&2748_{10}
\end{aligned}
$$

- Example
- Hexadecimal to decimal conversion:
  - **Convert $4AF_{16}$ to decimal**
  - **$4 \times 16^2 + A \times 16^1 + F \times 16^0$**
  - **$4 \times 16^2 + 10 \times 16^1 + 15 \times 16^0 = 1199_{10}$**

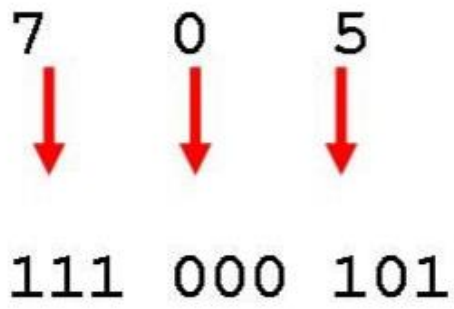
Octal to Binary



Technique

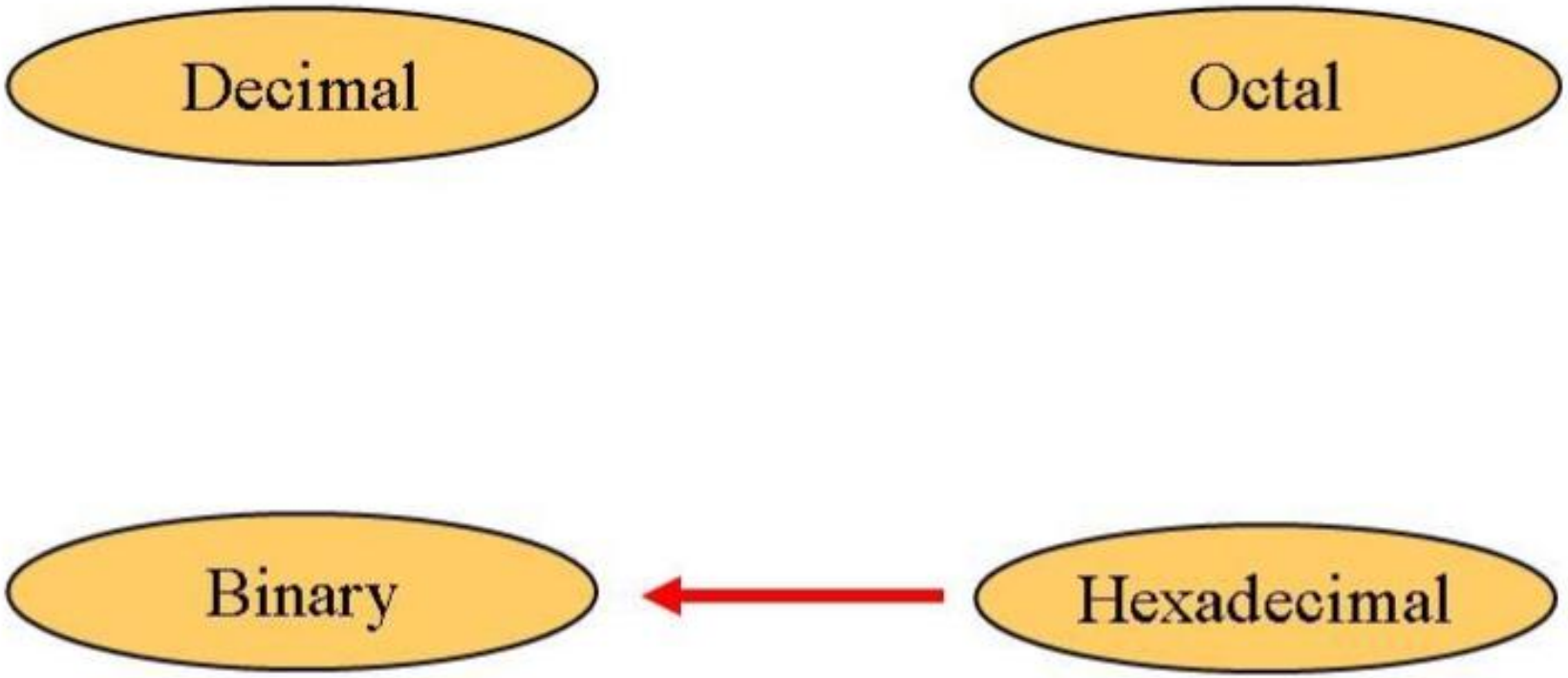
- Convert each octal digit to a 3-bit equivalent binary representation

$705_8 = ?_2$



$705_8 = 111000101_2$

Hexadecimal to Binary

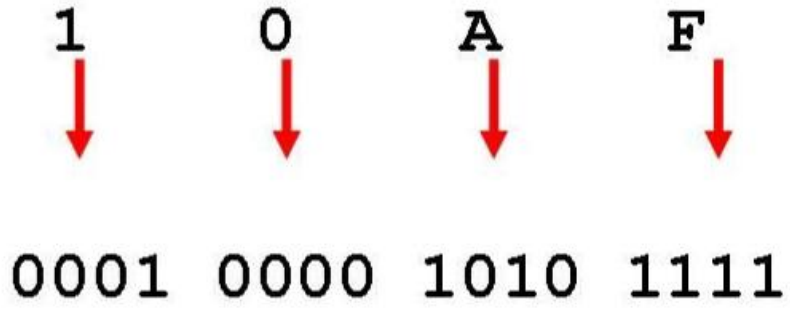


Hexadecimal to Binary Conversion

Technique

- Convert each hexadecimal digit to a 4-bit equivalent binary representation.

$10AF_{16} = ?_2$



$10AF_{16} = 0001000010101111_2$

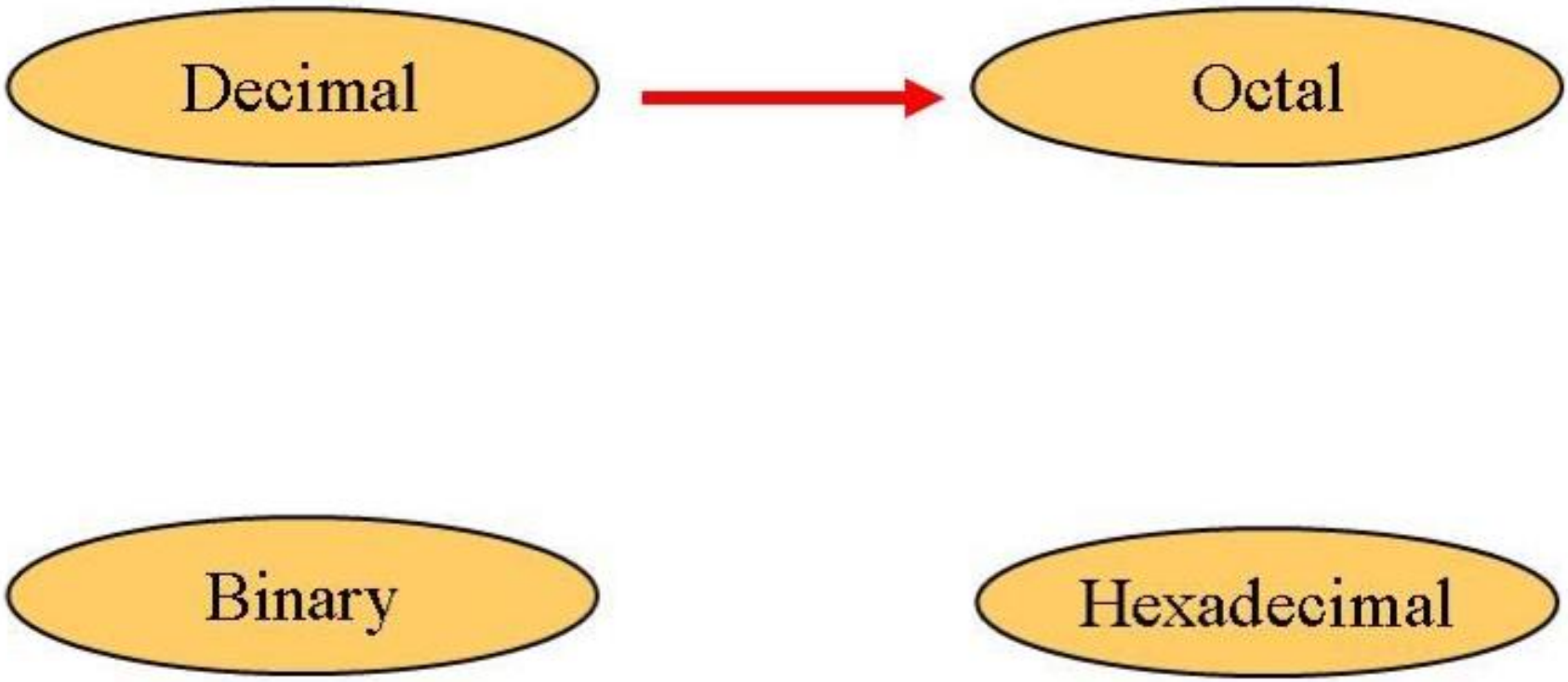
Hexadecimal to Binary Conversion

Technique

- Convert each hexadecimal digit to a 4-bit equivalent binary representation.

- Hexadecimal to binary conversion:
  - **Convert 4AF$_{16}$ (also written 0x4AF) to binary**
  - **0100 1010 1111**$_2$

Decimal to Octal

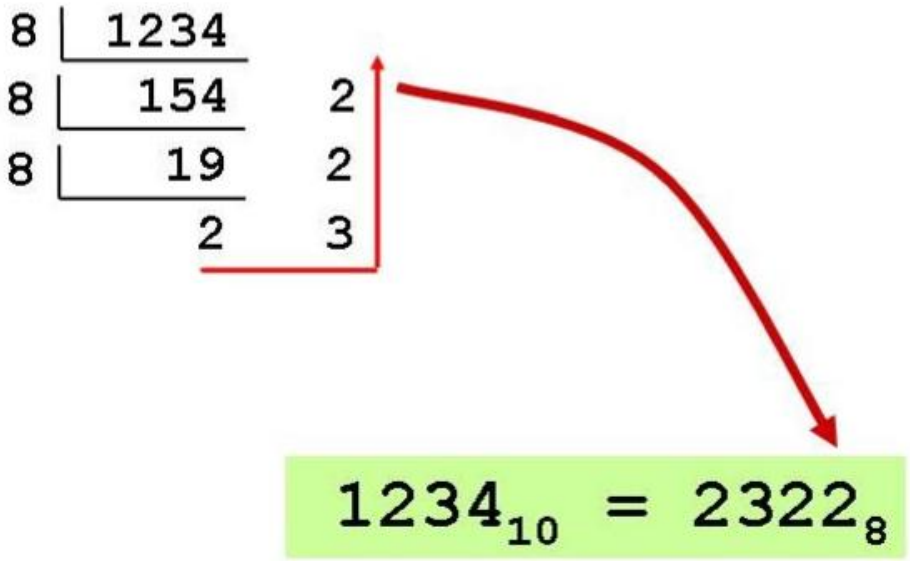


Deciam to Octal

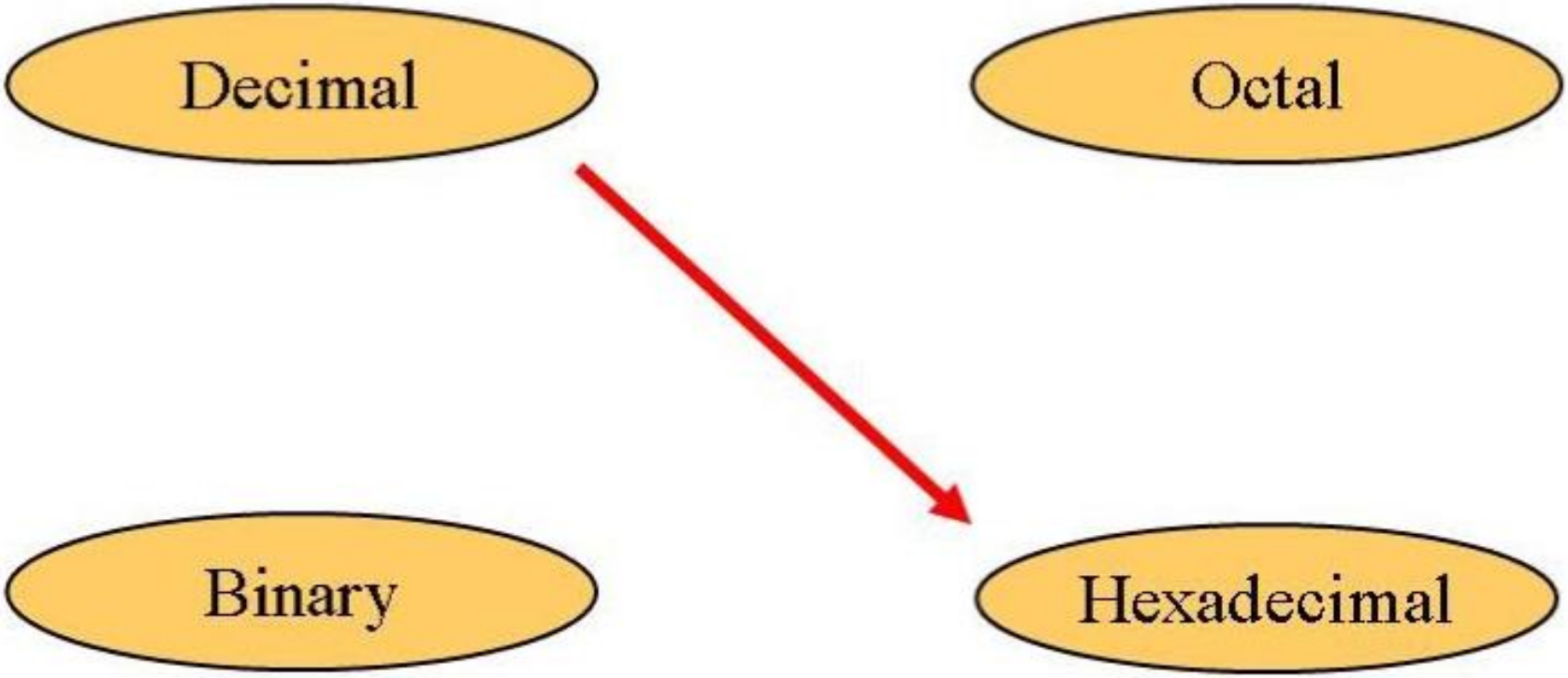
Technique

- Divide by 8
- Keep track of the remainder

$1234_{10} = ?_8$



Decimal to Hex

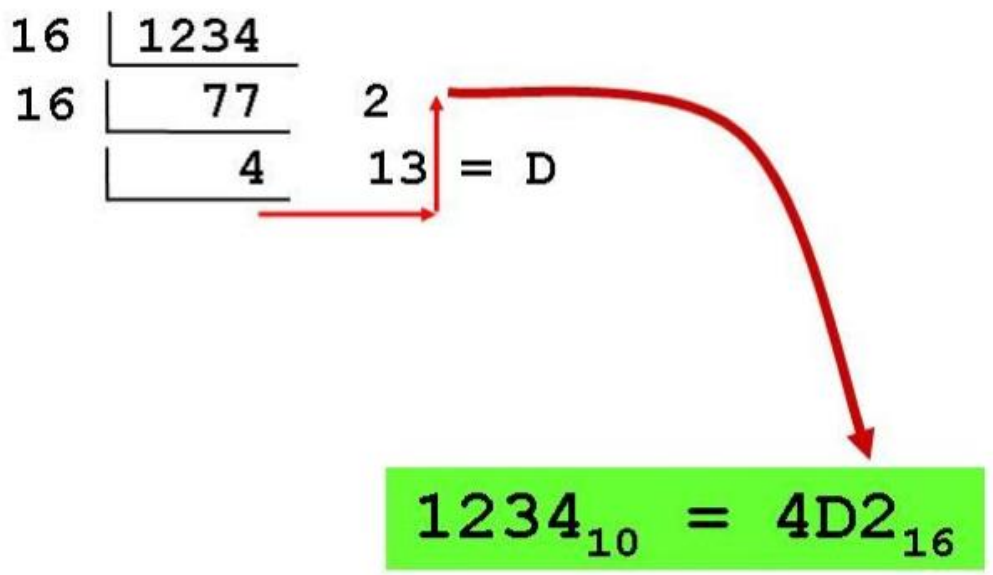


Decimal to Hex

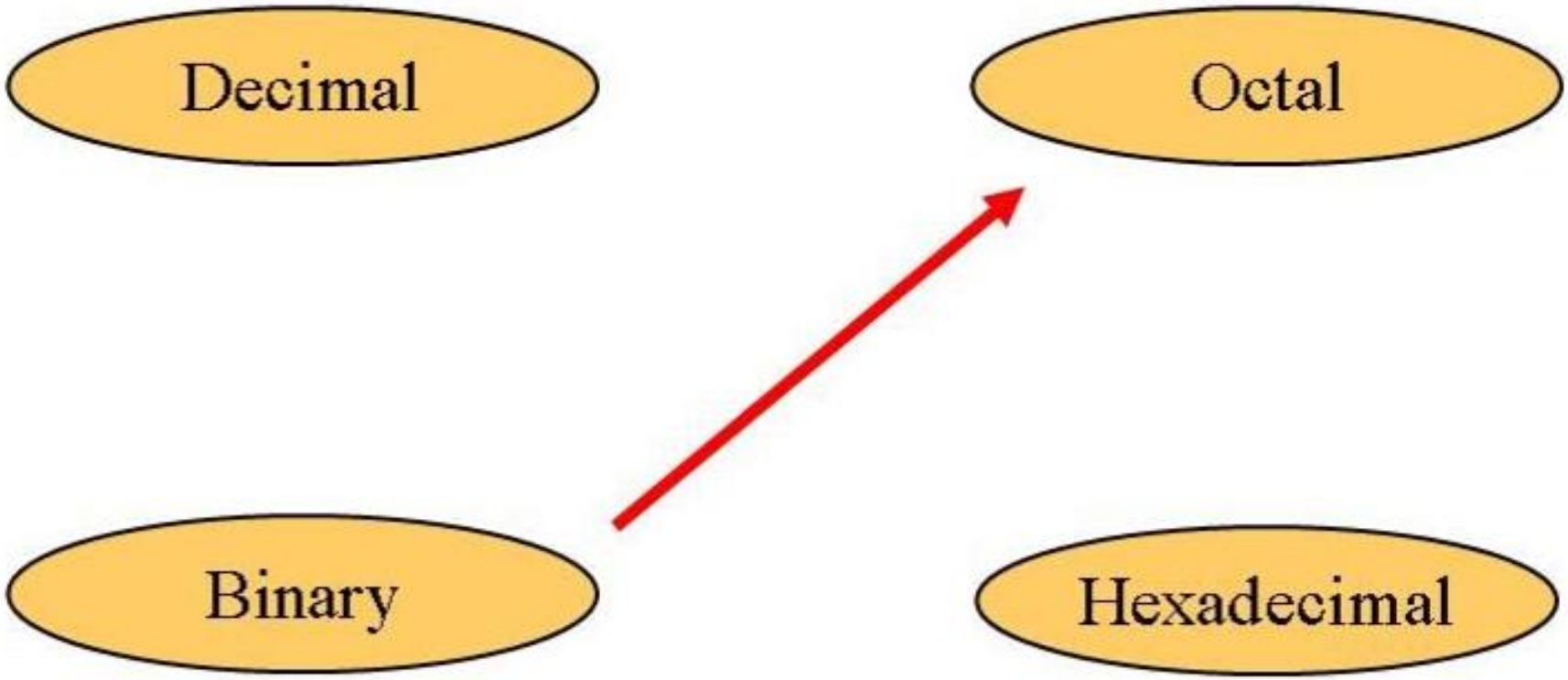
Technique

- Divide by 16
- Keep track of the remainder

$1234_{10} = ?_{16}$



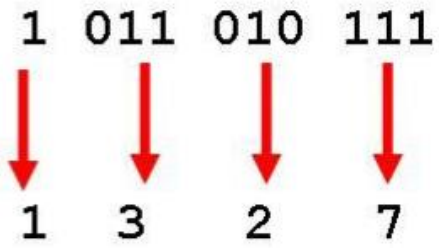
Binary to Octal



Technique

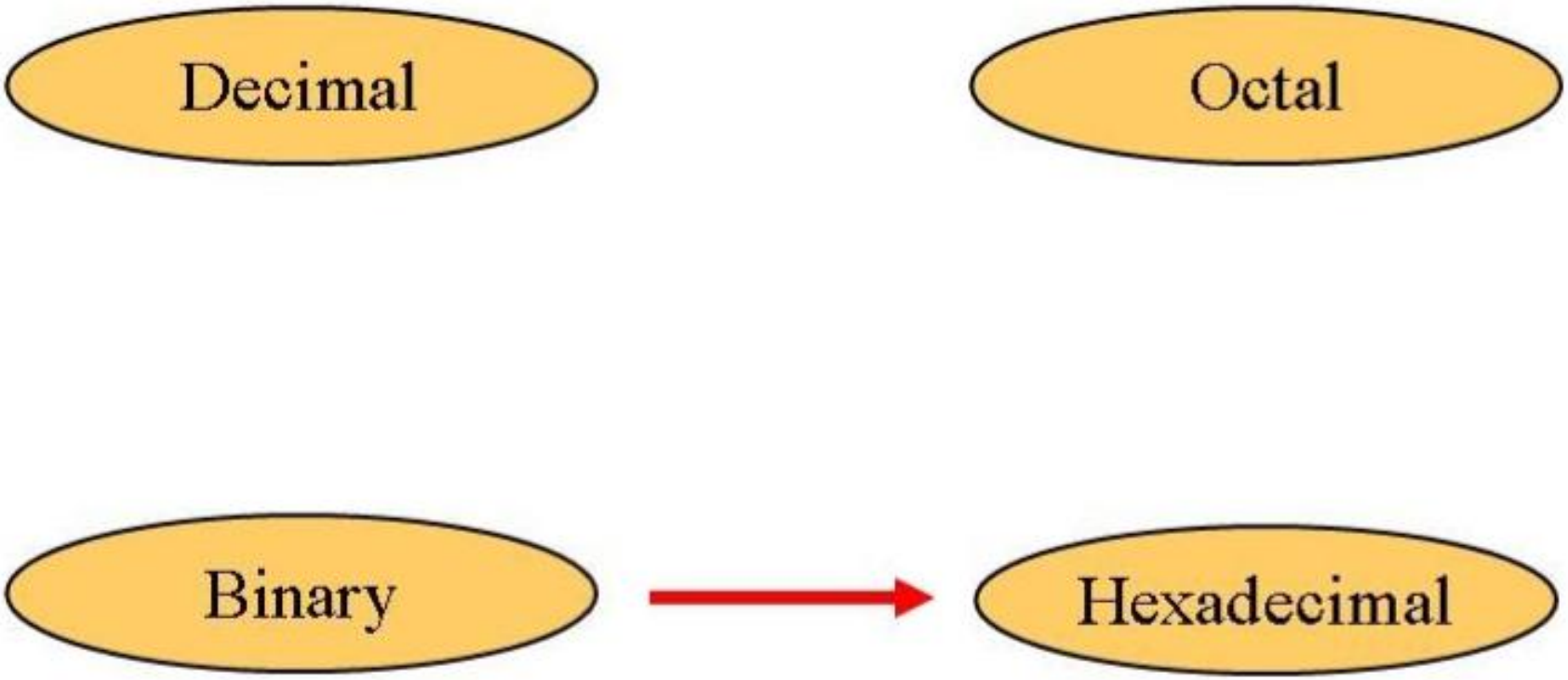
- Group bits in threes, starting on right
- Convert to octal digits

$$1011010111_2 = ?_8$$



$$1011010111_2 = 1327_8$$

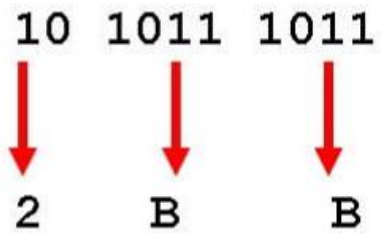
Binary to Hex



Technique

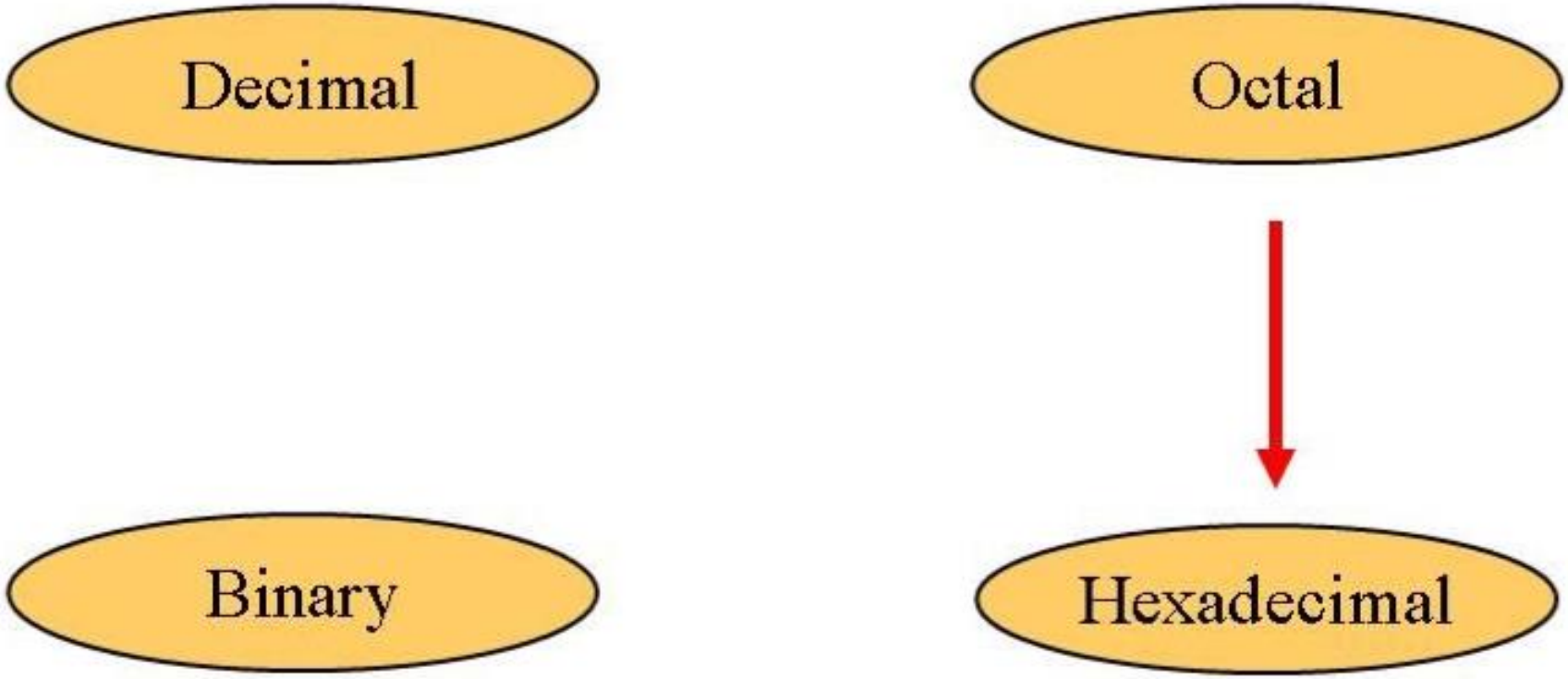
- Group bits in fours, starting on right
- Convert to hexadecimal digits

$1010111011_2 = ?_{16}$



$1010111011_2 = 2BB_{16}$

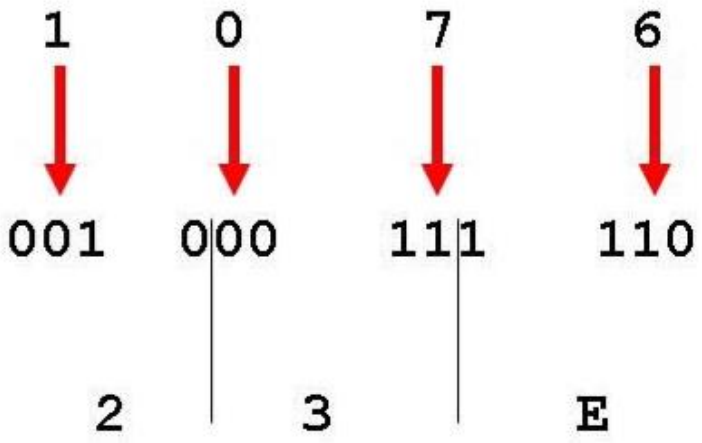
Octal to Hex



Technique

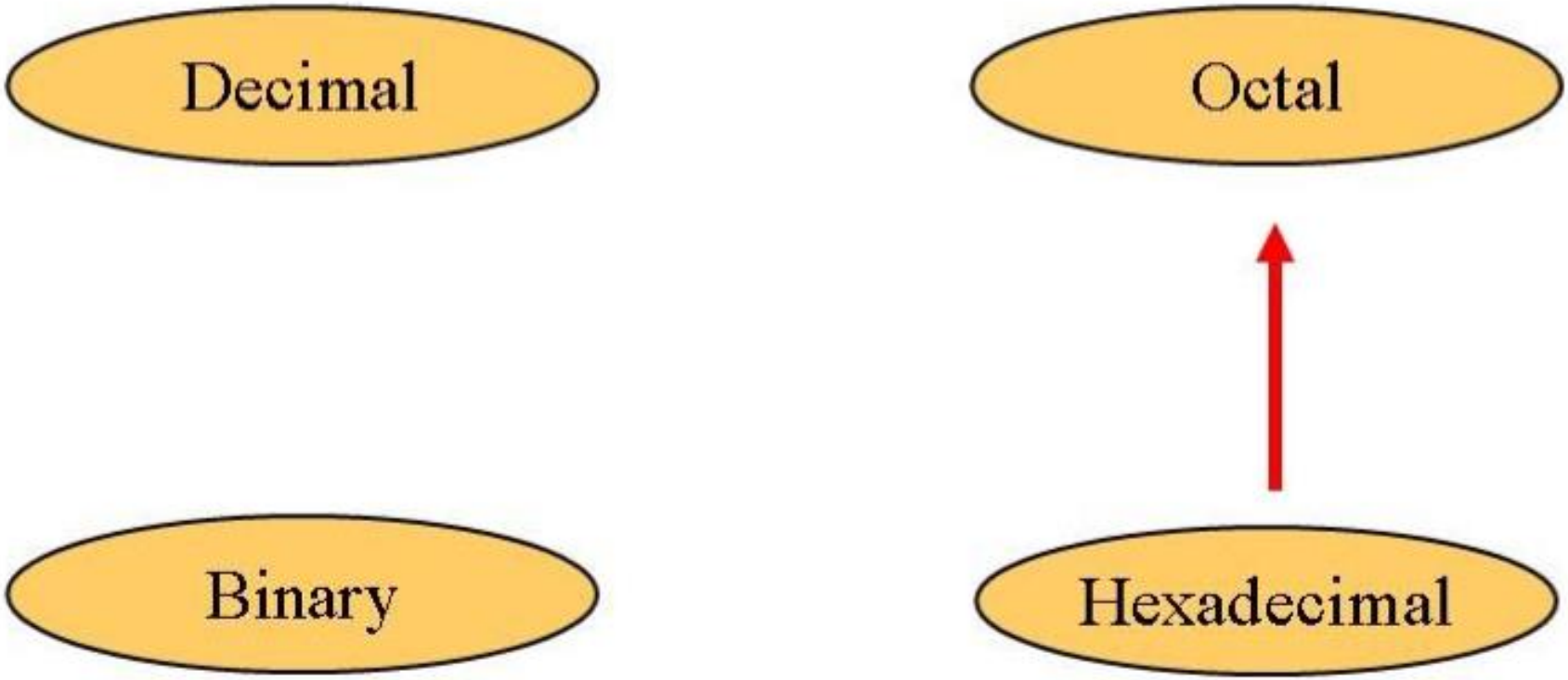
- Use binary as an intermediary

$1076_8 = ?_{16}$



$1076_8 = 23E_{16}$

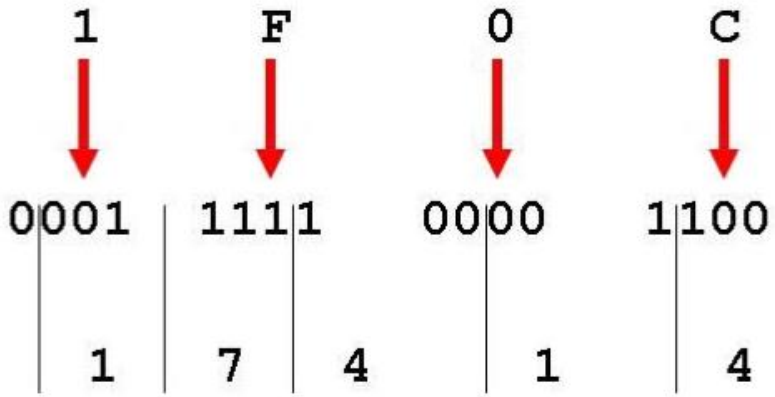
Hex to Octal



Technique

- Use binary as an intermediary

$1F0C_{16} = ?_8$



$1F0C_{16} = 17414_8$

Conversion Summary

	To				
From		Dec	Bin	Octal	Hex
	Dec	-	Repeated /	Repeated /	Repeated /
	Bin	Add weights of 1s	-	Group & convert 3-bits	Group & convert 4-bits
	Octal	Add digit*weight	Split each digit to 3-bits	-	Thru Bin
	Hex	Add digit*weights	Split each digit to 4-bits	Thru Bin	-

Exercise

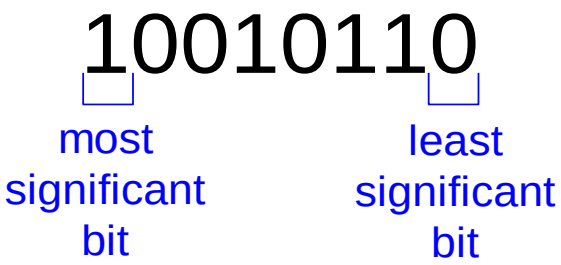
Decimal	Binary	Octal	Hexa-decimal
33			
	1110101		
		703	
			1AF

Answer

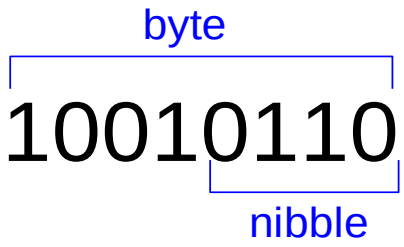
Decimal	Binary	Octal	Hexa-decimal
33	100001	41	21
117	1110101	165	75
451	111000011	703	1C3
431	110101111	657	1AF

Bits, Bytes, Nibbles…

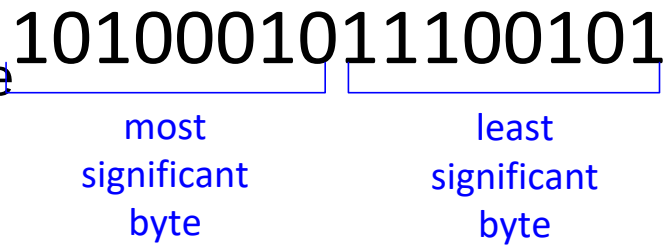
- **Bits**
  - **msb:** most significant bit
  - **lsb:** least significant bit



- **Bytes & Nibbles**

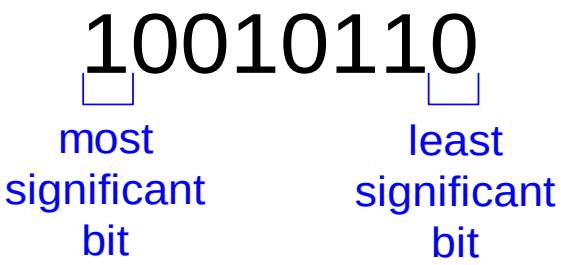


- **Bytes**
  - **MSB:** most significant byte
  - **LSB:** least significant byte

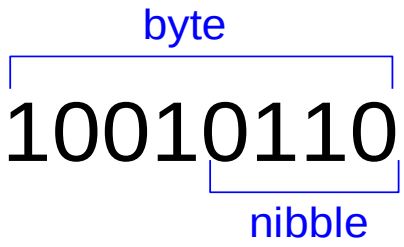


Bits, Bytes, Nibbles…

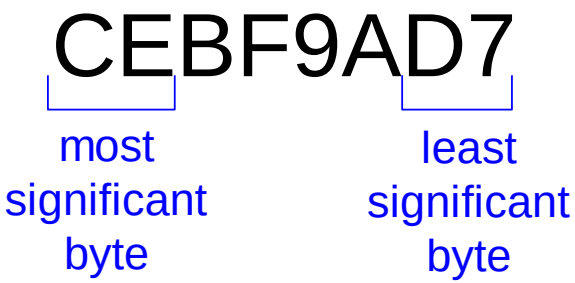
- **Bits**
  - **msb:** most significant bit
  - **lsb:** least significant bit



- **Bytes & Nibbles**



- **Bytes**
  - **MSB:** most significant byte
  - **LSB:** least significant byte
  - Each hex digit represents a nibble (4 bits)



Large Powers of Two

- 2^{10} = 1 kilo ≈ thousand (1024)
- 2^{20} = 1 mega ≈ million (1,048,576)
- 2^{30} = 1 giga ≈ billion (1,073,741,824)
- 2^{40} = 1 tera ≈ trillion (1,099,511,627,776)
- 2^{50} = 1 peta ≈ 10^{15}
- 2^{60} = 1 exa ≈ 10^{18}

Large Powers of Two

- 2^{10} = 1 kilo (**kibi**) ≈ 10^3 (1024)
- 2^{20} = 1 mega (**mebi**) ≈ 10^6 (1,048,576)
- 2^{30} = 1 giga (**gibi**) ≈ 10^9 (1,073,741,824)
- 2^{40} = 1 tera (**tebi**) ≈ 10^{12}
- 2^{50} = 1 peta (**pebi**) ≈ 10^{15}
- 2^{60} = 1 exa (**exbi**) ≈ 10^{18}

Large Powers of Two: Abbreviations

- 2^{10} = 1 kilo ≈ 1000 (1024)

  kibibyte = **1 Ki**

  for example: 1 KiB = 1024 Bytes

  1 Kib = 1024 bits

- 2^{20} = 1 mega ≈ 1 million (1,048,576)

  mebibyte = **1 Mi**

  for example: 1 MiB, 1 Mib (1 megabit)

- 2^{30} = 1 giga ≈ 1 billion (1,073,741,824)

  gibibyte = **1 Gi**

  for example: 1 GiB, 1 Gib

Estimating Powers of Two

- What is the approximate value of 2^{24}?

- Approximately how many values can a 32-bit variable represent?

Estimating Powers of Two

- What is the approximate value of 2^{24}?

  $2^4 \times 2^{20} \approx$ 16 million

- Approximately how many values can a 32-bit variable represent?

  $2^2 \times 2^{30} \approx$ 4 billion

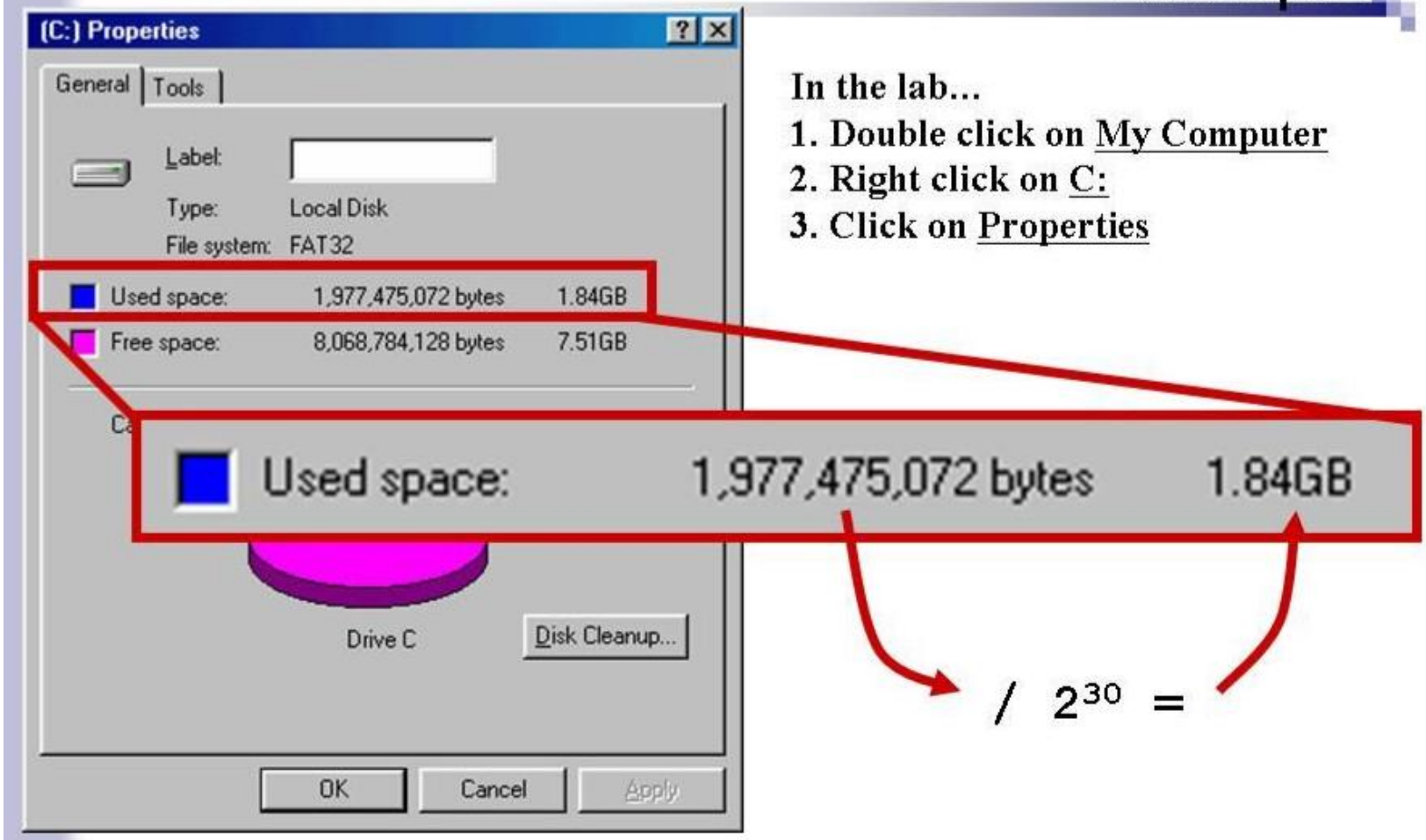
First factor out the largest 2^{10x}. Then estimate.

Common Powers of 2

Power	Preface	Symbol	Value
2^{10}	kilo	k	1024
2^{20}	mega	M	1048576
2^{30}	Giga	G	1073741824

- What is the value of "k", "M", and "G"?
- In computing, particularly w.r.t. memory, the base-2 interpretation generally applies

Example



In the lab…

1. **Double click on My Computer**
2. **Right click on C:**
3. **Click on Properties**

NEXT

Fractional Numbers Conversion